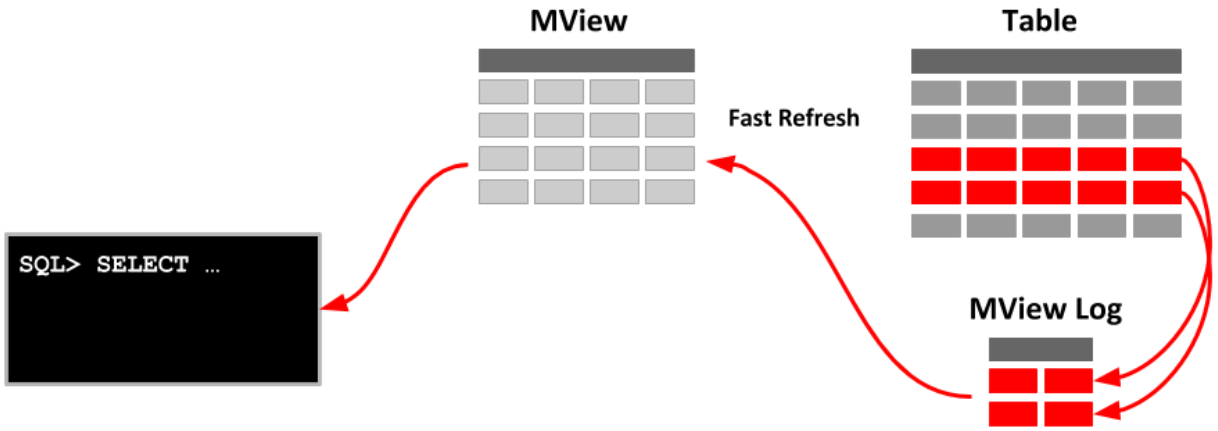
Materialized Views in Oracle

A materialized view, or snapshot as they were previously known, is a table segment whose contents are periodically refreshed based on a query, either against a local or remote table.

A materialized view in Oracle is a database object that contains the results of a query. They are local copies of data located remotely, or are used to create summary tables based on aggregations of a table's data. Materialized views, which store data based on remote tables are also, know as snapshots.



Using materialized views against remote tables is the simplest way to achieve replication of data between sites.

A materialized view can query tables, views, and other materialized views. Collectively these are called master tables (a replication term) or detail tables (a data warehouse term).

For replication purposes, materialized views allow you to maintain copies of remote data on your local node. These copies are read-only. If you want to update the local copies, you have to use the Advanced Replication feature. You can select data from a materialized view as you would from a table or view.

For data warehousing purposes, the materialized views commonly created are aggregate views, single-table aggregate views, and join views.

In replication environments, the materialized views commonly created are primary key, rowid, and subquery materialized views.

Primary Key Materialized Views

The following statement creates the primary-key materialized view on the table emp located on a remote database.

```
SQL>    CREATE MATERIALIZED VIEW mv_emp_pk
        REFRESH FAST START WITH SYSDATE
```

```
        NEXT  SYSDATE + 1/48
        WITH PRIMARY KEY
        AS SELECT * FROM emp@remote_db;
```

Materialized view created.

Note: When you create a materialized view using the FAST option you will need to create a view log on the master tables(s) as shown below:

```
SQL> CREATE MATERIALIZED VIEW LOG ON emp;
Materialized view log created.
```

Rowid Materialized Views

The following statement creates the rowid materialized view on table emp located on a remote database:

```
SQL>    CREATE MATERIALIZED VIEW mv_emp_rowid
        REFRESH WITH ROWID
        AS SELECT * FROM emp@remote_db;
```

Materialized view log created.

Subquery Materialized Views

The following statement creates a subquery materialized view based on the emp and dept tables located on the remote database:

```
SQL> CREATE MATERIALIZED VIEW  mv_empdept
AS SELECT * FROM emp@remote_db e
WHERE EXISTS
   (SELECT * FROM dept@remote_db d
   WHERE e.dept_no = d.dept_no)
```

REFRESH CLAUSE

```
[refresh [fast|complete|force]
        [on demand | commit]
        [start with date] [next date]
        [with {primary key|rowid}]]
```

The refresh option specifies:

a. The refresh method used by Oracle to refresh data in materialized view
b. Whether the view is primary key based or row-id based
c. The time and interval at which the view is to be refreshed

***Refresh Method* - FAST Clause**

The FAST refreshes use the materialized view logs (as seen above) to send the rows that have changed from master tables to the materialized view.

You should create a materialized view log for the master tables if you specify the REFRESH FAST clause.

```
SQL> CREATE MATERIALIZED VIEW LOG ON emp;

        Materialized view log created.
```

Materialized views are not eligible for fast refresh if the defined subquery contains an analytic function.

***Refresh Method* - COMPLETE Clause**

The complete refresh re-creates the entire materialized view. If you request a complete refresh, Oracle performs a complete refresh even if a fast refresh is possible.

***Refresh Method* - FORCE Clause**

When you specify a FORCE clause, Oracle will perform a fast refresh if one is possible or a complete refresh otherwise. If you do not specify a refresh method (FAST, COMPLETE, or FORCE), FORCE is the default.

PRIMARY KEY and ROWID Clause

WITH PRIMARY KEY is used to create a primary key materialized view i.e. the materialized view is based on the primary key of the master table instead of ROWID (for ROWID clause). PRIMARY KEY is the default option. To use the PRIMARY KEY clause you should have defined PRIMARY KEY on the master table or else you should use ROWID based materialized views.

Primary key materialized views allow materialized view master tables to be reorganized without affecting the eligibility of the materialized view for fast refresh.

Rowid materialized views should have a single master table and cannot contain any of the following:

- Distinct or aggregate functions
- GROUP BY Subqueries , Joins & Set operations

Timing the refresh

The START WITH clause tells the database when to perform the first replication from the master table to the local base table. It should evaluate to a future point in time. The NEXT clause specifies the interval between refreshes

```
SQL>    CREATE MATERIALIZED VIEW mv_emp_pk
        REFRESH FAST
        START WITH SYSDATE
        NEXT  SYSDATE + 2
        WITH PRIMARY KEY
```

AS SELECT * FROM emp@remote_db;

Materialized view created.

In the above example, the first copy of the materialized view is made at SYSDATE and the interval at which the refresh has to be performed is every two days.

A materialized view can be manually refreshed using the `DBMS_MVIEW` package.

```
EXEC DBMS_MVIEW.refresh('EMP_MV');
```

Check Privileges

Check the user who will own the materialized views has the correct privileges. At minimum they will require the `CREATE MATERIALIZED VIEW` privilege. If they are creating materialized views using database links, you may want to grant them `CREATE DATABASE LINK` privilege also.

```
CONNECT sys@db2

GRANT CREATE MATERIALIZED VIEW TO scott;
GRANT CREATE DATABASE LINK TO scott;
```

Create database link:

```
CREATE DATABASE LINK DB1.WORLD CONNECT TO scott IDENTIFIED BY tiger USING
'DB1.WORLD';
```

Summary

Materialized Views thus offer us flexibility of basing a view on Primary key or ROWID, specifying refresh methods and specifying time of automatic refreshes.

Reference:

https://www.databasejournal.com/features/oracle/article.php/10893_2192071_2/Materialized-Views-in-Oracle.htm

https://oracle-base.com/articles/misc/materialized-views